



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab experiment-13

Minimax Algorithm for Gaming

Aim

To write a Python program to implement the Minimax algorithm for a simple game.

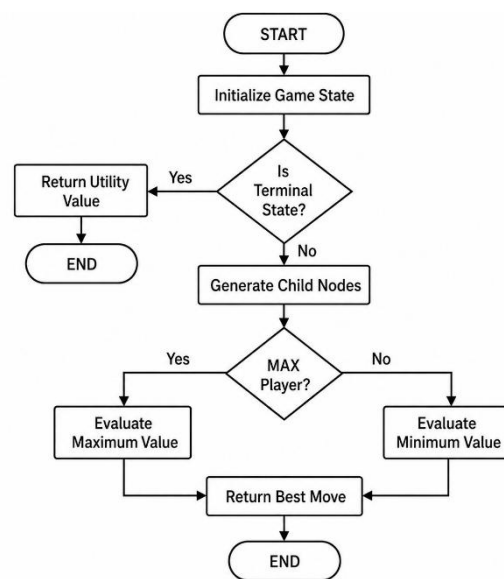
Objective

To understand the Minimax decision-making algorithm.

Algorithm

1. Start.
2. Initialize the game state.
3. Generate all possible moves.
4. Evaluate each possible move using Minimax.
5. Choose the move with the highest score for the AI.
6. Repeat until the game ends.
7. Display the winner.
8. Stop.

Flowchart



Code

```
def minimax(stones, ai):  
    if stones == 0:  
        return -1 if ai else 1  
    scores = []  
    for move in [1, 2]:  
        if move <= stones:  
            scores.append(minimax(stones - move, not ai))  
    return max(scores) if ai else min(scores)  
stones = int(input("Enter number of stones: "))  
while stones > 0:  
    print("Stones left:", stones)  
    # AI move  
    if stones <= 2:  
        move = stones
```

else:

```
    move = max([m for m in [1, 2] if m <= stones],
               key=lambda m: minimax(stones - m, False))
```

```
print("Computer takes:", move)
```

```
stones -= move
```

```
if stones == 0:
```

```
    print("Computer wins!")
```

```
    break
```

```
# Player move
```

```
move = int(input("Your move (1 or 2): "))
```

```
while move not in [1, 2] or move > stones:
```

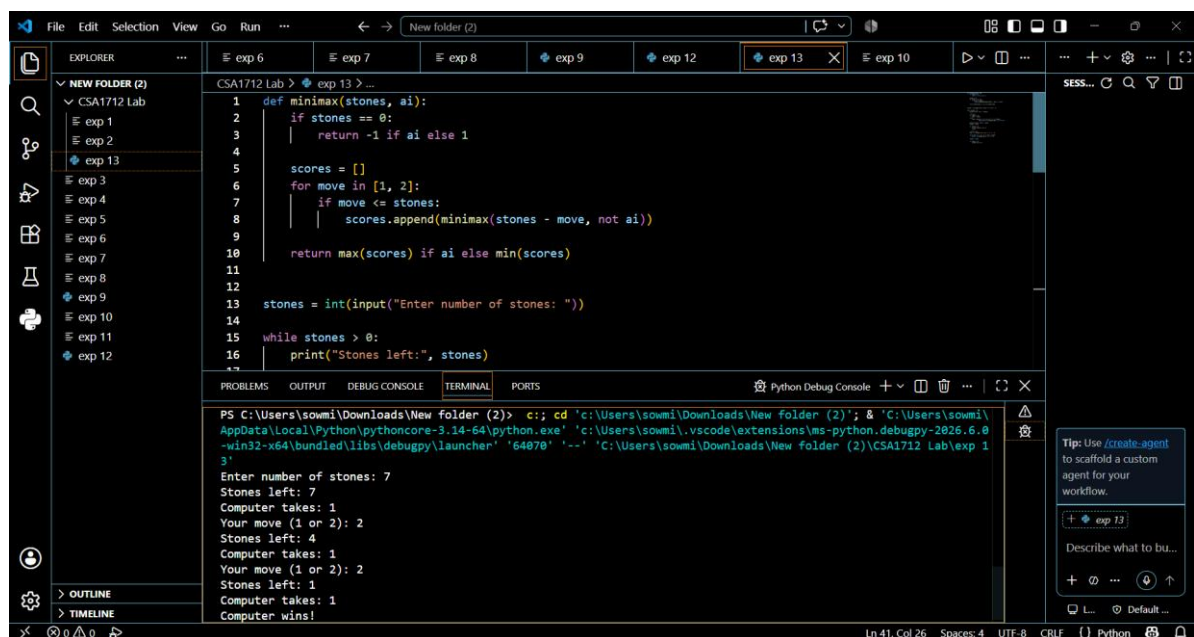
```
    move = int(input("Enter 1 or 2: "))
```

```
stones -= move
```

```
if stones == 0:
```

```
    print("You win!")
```

Output



The screenshot shows a Visual Studio Code editor with a Python file named 'exp 13'. The code implements a minimax algorithm for a game of stones. The terminal output shows the execution of the program, where the user enters the number of stones (7) and the game proceeds with alternating moves between the computer and the user until the computer wins.

```
1 def minimax(stones, ai):
2     if stones == 0:
3         return -1 if ai else 1
4
5     scores = []
6     for move in [1, 2]:
7         if move <= stones:
8             scores.append(minimax(stones - move, not ai))
9
10    return max(scores) if ai else min(scores)
11
12
13 stones = int(input("Enter number of stones: "))
14
15 while stones > 0:
16     print("Stones left:", stones)
```

```
PS C:\Users\sowmi\Downloads\New folder (2)> c:\; cd 'c:\Users\sowmi\Downloads\New folder (2)'; & 'C:\Users\sowmi\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\sowmi\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '64070' '--' 'C:\Users\sowmi\Downloads\New folder (2)\CSA1712 Lab\exp 13'
```

```
Enter number of stones: 7
Stones left: 7
Computer takes: 1
Your move (1 or 2): 2
Stones left: 4
Computer takes: 1
Your move (1 or 2): 2
Stones left: 1
Computer takes: 1
Computer wins!
```

Result

The game was implemented successfully using the Minimax algorithm.

Conclusion

The Minimax algorithm helps the computer select the best possible move by evaluating future game states.